

TECHNICAL DOCUMENTATION

PsychSupport

Student Mental Health Support Platform

Next.js 16 · TypeScript · Supabase · Tailwind CSS v4 · Radix UI

Version 0.1.0 · June 2026

Project Name	PsychSupport
Document Type	Technical Documentation
Version	0.1.0 (Private)
Tech Stack	Next.js 16.2 · React 19 · TypeScript 5.7 · Supabase · Tailwind CSS v4
Package Manager	pnpm
Target Deployment	Vercel (inferred from @vercel/analytics dependency)
Date	June 04, 2026
Audience	Developers, Architects, Technical Reviewers

Abstract This document provides a complete technical reference for the PsychSupport web application — an anonymous mental-health support platform built for university students. It covers system architecture, authentication model, database schema inferred from server actions, feature breakdown, component hierarchy, API surface, and configuration. All information is derived exclusively from source code and configuration files in the submitted archive.

Table of Contents

1 Project Overview

- 1.1 Purpose & Goals
- 1.2 Core Value Propositions
- 1.3 Tech Stack Summary

2 Repository Structure

- 2.1 Directory Layout
- 2.2 File Inventory

3 Architecture

- 3.1 Application Architecture
- 3.2 Rendering Strategy
- 3.3 Request Lifecycle
- 3.4 Data Flow Diagram

4 Authentication System

- 4.1 Authentication Modes
- 4.2 Anonymous User Flow
- 4.3 Registered User Flow
- 4.4 Session Management
- 4.5 OAuth Callback Route

5 Database Schema

- 5.1 profiles Table
- 5.2 therapists Table
- 5.3 therapist_availability Table
- 5.4 appointments Table
- 5.5 support_cards Table
- 5.6 Database Trigger

6 Feature Modules

- 6.1 Home — Tips on Calming Down
- 6.2 Anonymous Chat
- 6.3 Calendar & Appointment Booking
- 6.4 Support Materials Library
- 6.5 SOS Emergency Dialog

7 Server Actions API

- 7.1 auth.ts Actions
- 7.2 appointments.ts Actions

8 Component Architecture

- 8.1 Component Hierarchy
- 8.2 UI Component Library
- 8.3 Access-Control Pattern

9 Configuration & Build

- 9.1 Next.js Configuration
- 9.2 TypeScript Configuration
- 9.3 Tailwind CSS & PostCSS
- 9.4 shadcn/ui Configuration

9.5 Environment Variables

10 Security & Privacy Considerations

11 Analytics & Cookie Consent

12 Dependency Reference

1. Project Overview

1.1 Purpose & Goals

PsychSupport is a web-based mental health support application designed specifically for university students. Its primary goal is to lower the barrier to seeking psychological support by allowing completely anonymous access alongside optional full registration for richer features. The platform provides immediate self-help resources, a peer-support chat interface, therapist appointment scheduling, and curated psychoeducational materials — all within a single, cohesive interface.

1.2 Core Value Propositions

- **Anonymity-first:** Students can access calming techniques and crisis contacts without creating an account or revealing identity.
- **Graduated access:** Full features unlock upon registration (chat, appointments, materials), incentivising sign-up without blocking immediate help.
- **Crisis safety net:** The SOS Emergency dialog is always visible and accessible regardless of authentication state, providing direct phone numbers (116 123, 112) and a Text Support line (HOME to 741741).
- **Therapist marketplace:** A real-time calendar grid backed by a live Supabase database shows therapist availability with support for both online (video) and offline (in-person) sessions.
- **Evidence-based content:** All self-help techniques (Box Breathing, 5-4-3-2-1 Grounding, Progressive Muscle Relaxation, etc.) are labelled with their psychological basis.

1.3 Tech Stack Summary

Category	Technology	Version / Notes
Framework	Next.js	16.2.0 — App Router
UI Library	React	^19 (React 19)
Language	TypeScript	5.7.3, strict mode
Styling	Tailwind CSS	^4.2 (CSS-first config)
Component System	shadcn/ui + Radix UI	New-York style, full Radix suite
Backend / BaaS	Supabase	@supabase/ssr ^0.10, @supabase/supabase-js ^2.105
Auth	Supabase Auth	Email/password + anonymous sessions
Forms	React Hook Form + Zod	^7.54 + ^3.24
Date Handling	date-fns	4.1.0
Charts	Recharts	2.15.0
Notifications	Sonner	^1.7
Analytics	Vercel Analytics	1.6.1 (production-only)

Icons	Lucide React	^0.564
Package Manager	pnpm	lockfile present
Deployment Target	Vercel	Inferred from @vercel/analytics

2. Repository Structure

2.1 Directory Layout

The repository follows standard Next.js App Router conventions. All source code lives at the root level — there is no dedicated *src/* folder.

/ (root)	Project root — config files, middleware, package.json
app/	Next.js App Router — pages, layouts, server actions
app/page.tsx	Root page: auth-gate + session bootstrap
app/layout.tsx	Root layout: fonts, metadata, CookieConsentBanner, Analytics
app/globals.css	Global CSS: Tailwind imports, CSS custom properties
app/auth/callback/	OAuth callback route handler
app/actions/auth.ts	Server Actions: anonymous session, sign-in, register
app/actions/appointments.ts	Server Actions: therapist availability, booking, support cards
components/psych-support/	Domain components: auth, dashboard, sidebar, SOS, views
components/psych-support/views/	Full-page views: home, chat, calendar, materials
components/ui/	shadcn/ui primitives (~40 components)
components/cookie-consent-banner.tsx	GDPR cookie consent with localStorage persistence
components/theme-provider.tsx	next-themes wrapper
lib/supabase/	Supabase client factories: browser, server, middleware
lib/utils.ts	cn() utility (clsx + tailwind-merge)
hooks/	use-mobile.ts (breakpoint hook), use-toast.ts
middleware.ts	Edge middleware: session refresh on every request
public/	Static assets: icons, placeholder images
next.config.mjs	Next.js config: TS build-error bypass, unoptimised images
tsconfig.json	TypeScript strict config with @/* path alias
components.json	shadcn/ui registry config
postcss.config.mjs	@tailwindcss/postcss plugin

2.2 File Inventory

The submitted archive contains 111 files across the following categories:

Category	Count	Notes
Domain Components	5	auth-view, dashboard, sidebar, sos-dialog, plus 4 view components
shadcn/ui Primitives	~40	Full Radix-based component library
Server Actions	2	auth.ts (3 actions), appointments.ts (4 actions + 2 helpers)
Supabase Helpers	3	client.ts, server.ts, middleware.ts
Next.js Pages / Routes	2	page.tsx (root), auth/callback/route.ts
Configuration Files	6	next.config, tsconfig, postcss, components.json, .gitignore, pnpm-lock
CSS Files	2	app/globals.css, styles/globals.css
Public Assets	7	SVG icon, PNG icons (dark/light/apple), placeholder images
Hooks	2	use-mobile.ts, use-toast.ts

3. Architecture

3.1 Application Architecture

PsychSupport follows a **Single-Page Application within Next.js App Router** pattern. The root `page.tsx` acts as a client-side shell that conditionally renders either the `AuthView` or `Dashboard` based on session state. Navigation between the four main views (Home, Chat, Calendar, Materials) is managed through local React state — no Next.js routing is used for view transitions. Data fetching for authenticated operations uses Next.js Server Actions, keeping sensitive Supabase operations server-side.

3.2 Rendering Strategy

File	Rendering Mode	Responsibility
<code>app/page.tsx</code>	Client Component ('use client')	Auth state machine — manages session bootstrap, login, logout
<code>app/layout.tsx</code>	Server Component	Fonts, metadata, Analytics (prod-only), <code>CookieConsentBanner</code>
<code>app/auth/callback/route.ts</code>	Route Handler	Exchanges OAuth code for Supabase session
<code>components/psych-support/*</code>	Client Components	All domain components are client-side
<code>components/ui/*</code>	Client Components	<code>shadcn/ui</code> primitives
<code>app/actions/auth.ts</code>	Server Actions ('use server')	<code>signUp</code> , <code>signInWithPassword</code> via server-side Supabase
<code>app/actions/appointments.ts</code>	Server Actions ('use server')	Therapist queries, appointment insert, availability aggregation
<code>middleware.ts</code>	Edge Middleware	Runs on every request; refreshes Supabase session cookie

3.3 Request Lifecycle

1. Edge Middleware

`middleware.ts` intercepts every request (excluding static assets), calls `updateSession()` which creates a Supabase server client and invokes `supabase.auth.getUser()` to refresh the session token in cookies. Unauthenticated requests to `/protected/*` are redirected to `/auth/login`.

2. Root Page Bootstrap

`page.tsx` mounts and runs `checkSession()` via `supabase.auth.getSession()`. If a session exists, it fetches the profiles row (`anonymous_id`, `is_registered`) to determine access level. An `onAuthStateChanged` listener handles subsequent sign-in/sign-out events.

3. Conditional Render

Based on `isAuthenticated` state: `AuthView` is rendered for unauthenticated users; `Dashboard` is rendered for authenticated users, receiving `anonymousId`, `isRegistered`, and `onLogout` as props.

4. Server Action Invocation

Authenticated views (`CalendarView`, `ChatView`) call Server Actions directly. Server Actions instantiate a fresh server-side Supabase client per the Fluid Compute recommendation (no global client variable), execute database operations, and return typed result objects.

5. Database Operations

Supabase executes SQL against tables: `profiles`, `therapists`, `therapist_availability`, `appointments`, `support_cards`. Row-Level Security policies are assumed to be configured externally.

3.4 Data Flow Diagram

The following table summarises the key data flows in the application:

Flow	Path
Anonymous Login	Client → <code>saveAnonymousUserSession()</code> (Server Action) → <code>supabase.auth.signUp()</code> → <code>profiles</code> trigger → <code>profiles.anonymous_id</code> → Client
Registered Login	Client → <code>supabase.auth.signInWithPassword()</code> → <code>profiles</code> select → Dashboard
View Therapist Availability	<code>CalendarView</code> → <code>getAllAvailability()</code> → <code>therapist_availability</code> SELECT → aggregated slot map → Calendar grid
Book Appointment	<code>CalendarView</code> → <code>getAvailableTherapists()</code> → <code>createSupportCard()</code> → <code>createAppointment()</code> → <code>appointments</code> INSERT → Confirmation
Chat Message	<code>ChatView</code> → client-side sentiment analysis → response dictionary lookup → animated bot response (no backend call)
Materials Browse	<code>MaterialsView</code> → static in-component data (no backend call) → Dialog detail view
Session Refresh	Edge middleware → <code>supabase.auth.getUser()</code> → cookie update on every request

4. Authentication System

4.1 Authentication Modes

The application supports three authentication modes, all backed by Supabase Auth:

Mode	Mechanism	Initial Access	Feature Access
Anonymous	supabase.auth.signUp() with auto-generated email (anon-{ts}@anonymous.psychsupport.local) and random password	Home view (Calming Tips), SOS dialog	Limited — no chat, calendar, or materials
Registered	supabase.auth.signUp() with university email, first/last name, phone, 6-digit student ID	All features	Full access after email confirmation
Returning User	supabase.auth.signInWithPassword() with email + password	All features	Full access

4.2 Anonymous User Flow

1. User clicks **Generate Anonymous ID** in AuthView.
2. Client calls supabase.auth.signUp() with a timestamped synthetic email and random password. Both email and password are generated client-side; user never sees them.
3. Supabase creates an auth user with metadata is_registered: false.
4. A database trigger automatically creates a profiles row and generates a human-readable anonymous_id (e.g. ANON-XK7F2A).
5. Client fetches profiles.anonymous_id and calls onLogin(userId, false, anonymousId).
6. **Fallback:** If Supabase signup fails, a local anonymous ID (ANON-XXXXXX) is generated client-side and the user is placed in local-only anonymous mode.

4.3 Registered User Flow

Registration collects: first name, last name, email, phone number (validated via regex: international format), student ID (exactly 6 digits), and password (minimum 6 characters, confirmation required). Supabase sends an email confirmation link; the user must verify before gaining access. On successful signup the UI transitions to a 'Check Your Email' confirmation screen.

Note on emailRedirectTo: The registration action reads NEXT_PUBLIC_DEV_SUPABASE_REDIRECT_URL first, falling back to NEXT_PUBLIC_SITE_URL + /auth/callback. This supports local development override without modifying Supabase dashboard redirect URLs.

4.4 Session Management

Sessions are managed via cookies through `@supabase/ssr`. Three client factories handle different execution contexts:

File	Context	Behaviour
lib/supabase/client.ts	Browser (Client Component)	Singleton <code>createBrowserClient</code> — reuses single instance to prevent multiple <code>GoTrueClient</code> warnings
lib/supabase/server.ts	Server (Server Components / Actions)	<code>createServerClient</code> per-call; reads/writes cookies via <code>next/headers</code> . Must be re-created on every invocation (Fluid Compute compatible).
lib/supabase/middleware.ts	Edge Runtime	<code>createServerClient</code> per-request; synchronises cookie state between browser and Supabase on every navigation.

4.5 OAuth Callback Route

The route handler at `app/auth/callback/route.ts` implements the PKCE code exchange. It reads the code query parameter, calls `supabase.auth.exchangeCodeForSession(code)`, and redirects to the next parameter (defaulting to `/`) on success, or to `/auth/error` on failure.

5 Database Schema

Note: No migration files or schema SQL are present in the submitted archive. The schema below is inferred entirely from Supabase query calls in server actions and client code. Column types are best-estimate from usage context.

5.1 profiles Table

Stores user profile data. One row per Supabase auth user.

Column	Type	Nullable	Notes
id	uuid	NOT NULL	Foreign key → auth.users.id (primary key)
anonymous_id	text	nullable	Human-readable ID (e.g. ANON-XK7F2A). Generated by DB trigger on insert.
is_registered	boolean	NOT NULL	false for anonymous users, true for registered users
first_name	text	nullable	Populated from auth metadata on registration
last_name	text	nullable	Populated from auth metadata on registration

5.2 therapists Table

Master list of therapists available for booking.

Column	Type	Nullable	Notes
id	uuid	NOT NULL	Primary key
first_name	text	NOT NULL	
last_name	text	NOT NULL	
title	text	nullable	e.g. 'Dr.', 'MSc.'
specializations	text[]	NOT NULL	Array of specialisation strings
bio	text	nullable	Short biography
avatar_url	text	nullable	URL to profile image
is_online_available	boolean	NOT NULL	Supports video sessions
is_offline_available	boolean	NOT NULL	Supports in-person sessions

office_location	text	nullable	Physical office address
rating	numeric	NOT NULL	Average rating (e.g. 4.8)
total_reviews	integer	NOT NULL	Number of reviews
status	text	NOT NULL	Filtered by 'active' in queries

5.3 therapist_availability Table

Recurring weekly availability slots per therapist.

Column	Type	Nullable	Notes
id	uuid	NOT NULL	Primary key
therapist_id	uuid	NOT NULL	Foreign key → therapists.id
day_of_week	text	NOT NULL	Enum: monday, tuesday, ..., saturday (lowercase)
start_time	time	NOT NULL	24-hour format, e.g. '09:00'
end_time	time	NOT NULL	24-hour format, e.g. '17:00'
meeting_type	text	NOT NULL	Enum: 'online' 'offline'
slot_duration_minutes	integer	NOT NULL	Typically 60 minutes
is_available	boolean	NOT NULL	Filtered true in availability queries

5.4 appointments Table

Individual booked sessions between a user and therapist.

Column	Type	Nullable	Notes
id	uuid	NOT NULL	Primary key, returned after INSERT
user_id	uuid	NOT NULL	Foreign key → auth.users.id
therapist_id	uuid	NOT NULL	Foreign key → therapists.id

appointment_date	date	NOT NULL	YYYY-MM-DD format
start_time	time	NOT NULL	HH:MM 24-hour
end_time	time	NOT NULL	Calculated as start_time + 1 hour
meeting_type	text	NOT NULL	'online' 'offline'
status	text	NOT NULL	Default: 'scheduled'
meeting_link	text	nullable	Generated as https://meet.psychsupport.edu/session/{uuid[:8]} for online sessions
office_address	text	nullable	Copied from therapist.office_location for offline sessions
notes	text	nullable	User-provided notes from support card or consultant form

5.5 support_cards Table

Pre-appointment mood and context cards filled by the user before an offline session.

Column	Type	Nullable	Notes
id	uuid	NOT NULL	Primary key
user_id	uuid	NOT NULL	Foreign key → auth.users.id
title	text	NOT NULL	Auto-generated: 'Session with {title} {first} {last}'
current_mood	integer	NOT NULL	Slider value 1–10
stress_level	integer	NOT NULL	Slider value 1–10
recent_challenges	text	NOT NULL	Free-text from user
support_goals	text	NOT NULL	Free-text from user
additional_notes	text	nullable	Optional free-text
is_active	boolean	NOT NULL	Default: true

5.6 Database Trigger

The `anonymous_id` in the `profiles` table is generated by a PostgreSQL database trigger that fires on INSERT into the `profiles` table. This is evidenced by the server action fetching `profiles.anonymous_id` after `signUp()` completes — the value is never set by the application code directly. The trigger format produces IDs matching the pattern `ANON-XXXXXX` (uppercase alphanumeric suffix).

6

6. Feature Modules

6.1 Home — Tips on Calming Down

Component: components/psych-support/views/home-view.tsx | Access: All users (anonymous and registered)

The Home view displays 8 evidence-based calming technique cards in a responsive grid (1 → 2 → 4 columns). Each card uses a distinct colour theme and icon. Clicking a card opens a Modal dialog with step-by-step instructions and a 'Pro tip' callout explaining the psychological mechanism.

Technique	Psychological Basis
Box Breathing	4-4-4-4 pattern; activates parasympathetic nervous system
5-4-3-2-1 Grounding	Sensory anchoring to counter dissociation/anxiety
Progressive Muscle Relaxation	Sequential tension-release from feet to forehead
Mindful Listening	Non-judgmental awareness of environmental sounds
Self-Compassion	Kristin Neff-style self-kindness practice
Safe Place Visualisation	Guided imagery to create a mental refuge
Sleep Hygiene	Environment and routine optimisation for rest
Wave Breathing	Extended exhale (5:7 ratio) to stimulate vagus nerve

6.2 Anonymous Chat

Component: components/psych-support/views/chat-view.tsx | Access: Registered users only

The Chat module provides a simulated peer-support chatbot. All processing is client-side — no backend AI API is called. The bot uses a **keyword-based sentiment detection engine** to classify user messages and select contextually appropriate responses from pre-defined dictionaries.

Sentiment Classification Logic:

Sentiment	Detection Rule	Response Behaviour
crisis	Highest priority. Triggered by words: suicide, self-harm, cutting, hurt, die, end it, give up, can't go on. Bypasses all other scoring.	Red indicator; crisis resources shown
positive	Keyword count comparison: positive word hits vs negative vs inquiry.	Celebratory responses; upbeat quick replies
negative	Negative word count dominates.	Empathetic validation; coping resource offers
inquiry	Question words and help-seeking phrases dominate.	Informational responses

neutral	No clear majority.	Open-ended engagement
---------	--------------------	-----------------------

Responses are selected to avoid repetition using a rotation tracker. Quick reply chips update after each bot turn. A simulated typing indicator (animated dots) is shown for 1–2.5 seconds. The user's anonymousId is displayed in the chat header but is not transmitted to any backend.

6.3 Calendar & Appointment Booking

Component: `components/psych-support/views/calendar-view.tsx` | **Access:** Registered users only

The Calendar view renders a weekly availability grid (Monday–Saturday × 8:00 AM–8:00 PM) populated from live Supabase data via `getAllAvailability()`. Cells are colour-coded: blue for online slots, green for offline (in-person) slots. The booking flow proceeds through three multi-step dialog states:

- **Step 1: select-therapist:** On slot click: `getAvailableTherapists(day, time, type)` is called. Available therapists are listed with name, title, specialisations, rating, bio, and meeting type badges. User selects a therapist to proceed.
- **Step 2: form:** For offline bookings: a Support Card form collects current mood (slider 1–10), stress level (slider 1–10), recent challenges (textarea), support goals (textarea), and optional additional notes. For online bookings: a consultant form collects topic, preferred language, and notes.
- **Step 3: confirm:** `createSupportCard()` and `createAppointment()` are called via Server Actions. On success, a confirmation card displays the therapist name, date, time, meeting type, and location/link.

Date Calculation: `getNextDateForDay(dayName)` computes the next calendar date for the selected weekday name. Time conversion helpers `convertTo24Hour()` and `convertTo12Hour()` handle the 12 ↔ 24 hour format translation needed between the UI and the database.

6.4 Support Materials Library

Component: `components/psych-support/views/materials-view.tsx` | **Access:** Registered users only

The Materials view presents a searchable, filterable library of psychoeducational articles. All content is static (embedded in the component); no external CMS or database call is made. Users can search by title/description and filter by category badge.

Article Title	Category	Read Time
Understanding Anxiety: A Student's Guide	Anxiety	8 min
Building Resilience Through Self-Care	Self-Care	6 min
Sleep Hygiene for Better Mental Health	Sleep	5 min
Navigating Social Relationships	Relationships	7 min
Managing Academic Stress	Stress	~6 min
Mindfulness for Beginners	Mindfulness	~5 min

6.5 SOS Emergency Dialog

Component: components/psych-support/sos-dialog.tsx | **Access:** ALL users (including unauthenticated guests)

The SOS button is always visible in the sidebar footer and triggers a full-screen modal with a red colour scheme. It provides:

- Crisis Hotline: **116 123** with a direct 'Call Now' button (tel: URI)
- Emergency Services: **112**
- University Counselling: contact your institution
- Text Support: Text HOME to **741741**
- An empathetic reassurance message

Crisis Detection in Chat: The ChatView independently monitors all messages for crisis-indicator keywords. On detection, the sentiment is set to 'crisis', a red visual indicator is shown on the message, and the bot response shifts to crisis-support mode — prompting professional help and the SOS dialog.

7. Server Actions API

Server Actions are the primary data-mutation and sensitive-query mechanism. They execute exclusively on the server, use the server-side Supabase client, and return typed result objects. All actions handle errors and return `{ success: false, error: string }` on failure.

7.1 auth.ts Actions

`saveAnonymousUserSession()`

Parameters	none
Returns	Promise
Description	Generates a synthetic email/password pair, calls <code>supabase.auth.signUp()</code> with <code>is_registered: false</code> metadata, then fetches the DB-generated <code>anonymous_id</code> . Returns <code>{ success, anonymousId }</code> or fallback ID if profile fetch fails.

`signInUser(email, password)`

Parameters	email: string, password: string
Returns	Promise< <code>{ success, userId, isRegistered, anonymousId, firstName, lastName }</code> >
Description	Wraps <code>supabase.auth.signInWithPassword()</code> and fetches the profiles row to return full profile context.

`registerUser(email, password, firstName, lastName, phone, studentId)`

Parameters	6 string params
Returns	Promise< <code>{ success, userId, emailConfirmationRequired }</code> >
Description	Calls <code>supabase.auth.signUp()</code> with full user metadata. Reads <code>NEXT_PUBLIC_DEV_SUPABASE_REDIRECT_URL</code> for local dev override of email redirect URL.

7.2 appointments.ts Actions

`getAllAvailability()`

Parameters	none
Returns	Promise< <code>{ success, data?: { day, time, type, count }[] }</code> >
Description	Fetches all <code>therapist_availability</code> rows where <code>is_available=true</code> . Iterates time ranges hour by hour, building a slot map keyed by day-time. Returns aggregated slot counts used to populate the calendar grid colour coding.

`getAvailableTherapists(dayOfWeek, timeSlot, meetingType)`

Parameters	string, string, 'online' 'offline'
Returns	Promise<{ success, data?: AvailableSlot[] }>
Description	Converts timeSlot from 12-hour to 24-hour, queries therapist_availability for matching day/type/time window, then JOINS to therapists (status='active'). Returns combined AvailableSlot[] objects.

`createAppointment(therapistId, date, startTime, endTime, type, notes?)`

Parameters	uuid, date, time, time, type, optional string
Returns	Promise<{ success, appointmentId? }>
Description	Verifies authentication, fetches therapist office_location, inserts into appointments with auto-generated meeting_link (online) or office_address (offline). Returns the new appointment UUID.

`createSupportCard(...)`

Parameters	title, mood, stress, challenges, goals, notes?
Returns	Promise<{ success, supportCardId? }>
Description	Verifies authentication, inserts into support_cards with is_active=true. Called before createAppointment() in the offline booking flow.

8

8. Component Architecture

8.1 Component Hierarchy

RootLayout (Server)	app/layout.tsx	Fonts · CookieConsentBanner · Analytics
PsychSupportApp (Client)	app/page.tsx	Auth state machine; renders AuthView or Dashboard
AuthView (Client)	components/psych-support/auth-view.tsx	Login tab · Register tab · Anonymous ID generation
Dashboard (Client)	components/psych-support/dashboard.tsx	Sidebar + main content + SOS dialog orchestration
Sidebar (Client)	components/psych-support/sidebar.tsx	Nav items · user status badge · SOS button · logout
HomeView (Client)	components/psych-support/views/home-view.tsx	Calming tips grid + tip detail dialog
CalendarView (Client)	components/psych-support/views/calendar-view.tsx	Availability grid + multi-step booking dialogs
ChatView (Client)	components/psych-support/views/chat-view.tsx	Message list · sentiment engine · quick replies · input
MaterialsView (Client)	components/psych-support/views/materials-view.tsx	Search · category filter · article cards · detail dialog
SosDialog (Client)	components/psych-support/sos-dialog.tsx	Emergency contacts · crisis resources
RestrictedView (inline)	Inside Dashboard	Lock icon + sign-out prompt for unauthenticated feature access
Toaster (Client)	components/ui/sonner.tsx	Toast notification host
CookieConsentBanner (Client)	components/cookie-consent-banner.tsx	GDPR consent · localStorage persistence

8.2 UI Component Library

The project uses shadcn/ui with the 'new-york' style variant and neutral base colour. Components are copied into the repository (not installed as a package) under components/ui/. All components use Radix UI primitives for accessibility.

Group	Components
Layout / Navigation	Sidebar, Breadcrumb, NavigationMenu, Menubar, Tabs, ResizablePanels
Overlay / Feedback	Dialog, AlertDialog, Sheet, Drawer, Popover, HoverCard, Tooltip, Toast (Sonner), Alert
Form Controls	Button, Input, Textarea, Select, Checkbox, RadioGroup, Switch, Slider, InputOTP

Data Display	Table, Card, Badge, Avatar, Accordion, Carousel, Chart (Recharts wrapper), Progress
Command / Search	Command (cmdk), DropdownMenu, ContextMenu
Utility	ScrollArea, Separator, Skeleton, Spinner, Empty, Field, Item, Kbd, Label

8.3 Access-Control Pattern

Feature access is enforced at the component level via the `isRegistered` boolean prop passed down from the root `PsychSupportApp`. The pattern is consistent across the application:

Location	Access Control Mechanism
Sidebar nav items	Lock icon shown on restricted items; clicking still navigates (then blocked at view level)
Header	Guest Access badge shown when <code>!isRegistered</code>
<code>Dashboard.renderView()</code>	Restricted views render <code>RestrictedView</code> component instead of the real view
<code>RestrictedView</code>	Displays a Lock icon card with 'Sign Out & Register' button
SOS Dialog	Always rendered, always accessible regardless of auth state

9. Configuration & Build

9.1 Next.js Configuration (next.config.mjs)

Option	Value	Notes
typescript.ignoreBuildErrors	true	TypeScript errors do not fail production builds. This is a development convenience flag and should be reviewed before production hardening.
images.unoptimized	true	Disables Next.js Image Optimization API. All components use unoptimised delivery. Suitable for static exports or Vercel-free-tier deployments.

9.2 TypeScript Configuration (tsconfig.json)

Option	Value	Notes
target	ES6	
module	esnext	
moduleResolution	bundler	Next.js bundler-mode resolution
strict	true	Full strict type-checking enabled
jsx	react-jsx	
incremental	true	Incremental compilation cache
paths	@/* → ./*	Root-relative import alias
plugins	next	Next.js TypeScript plugin
lib	dom, dom.iterable, esnext	

9.3 Tailwind CSS & PostCSS

Tailwind CSS v4 is used with the **CSS-first configuration model**. There is no `tailwind.config.js` — all design tokens are defined as CSS custom properties in `app/globals.css`. PostCSS is configured solely with the `@tailwindcss/postcss` plugin. The `tw-animate-css` package (1.3.3) provides animation utilities. CSS variables follow the `shadcn/ui` convention: `--background`, `--foreground`, `--primary`, `--primary-foreground`, `--sidebar-*`, etc.

9.4 shadcn/ui Configuration (components.json)

Key	Value	Notes
style	new-york	Tighter spacing, rounded-md corners

rsc	true	React Server Component support
tsx	true	TypeScript JSX
tailwind.baseColor	neutral	
tailwind.cssVariables	true	Uses CSS custom properties for theming
iconLibrary	lucide	lucide-react for all icons
aliases.components	@/components	
aliases.utils	@/lib/utils	

9.5 Environment Variables

The following environment variables are referenced in source code. They must be configured in the deployment environment (e.g. Vercel dashboard) or a `.env.local` file (excluded from version control via `.gitignore`).

Variable	Status	Exposure	Purpose
NEXT_PUBLIC_SUPABASE_URL	Required	Public	Supabase project URL
NEXT_PUBLIC_SUPABASE_ANON_KEY	Required	Public	Supabase anonymous/public API key
NEXT_PUBLIC_SITE_URL	Required for prod	Public	Base URL for email redirect: <code>{SITE_URL}/auth/callback</code>
NEXT_PUBLIC_DEV_SUPABASE_REDIRECT_URL	Optional	Public	Local dev override for email confirmation redirect URL

10. Security & Privacy Considerations

Server-Side Data Access

All database mutations and sensitive queries are performed in Server Actions (app/actions/). The Supabase anon key is used server-side with user context; actual data access control relies on Row-Level Security (RLS) policies configured in the Supabase dashboard — these policies are not visible in the source code but are essential for security.

Anonymous Identity Generation

Anonymous user emails follow the pattern anon-{timestamp}-{random}@anonymous.psychsupport.local. The domain 'anonymous.psychsupport.local' is a non-routable internal domain, reducing risk of accidental email delivery. Passwords are randomly generated and never shown to the user. The user only ever sees their anonymous_id (e.g. ANON-XK7F2A).

Session Security

Sessions are stored in HTTP-only cookies managed by @supabase/ssr. The Edge middleware refreshes sessions on every request, preventing silent session expiry. The middleware's supabase.auth.getUser() call (not getSession()) validates the JWT against Supabase servers rather than trusting the local cookie value.

TypeScript Build Errors Bypassed

next.config.mjs sets typescript.ignoreBuildErrors: true. This means TypeScript type errors will not prevent deployment. This is a significant risk for production systems and should be disabled before production hardening.

Crisis Content Safety

The ChatView monitors all messages for crisis keywords (suicide, self-harm, cutting, etc.) as a first-pass detection layer. Crisis responses include emergency numbers. This is not a clinical-grade safety system and should be supplemented with professional human moderation for a production mental health application.

Cookie Consent

The CookieConsentBanner implements GDPR-style consent with three categories: necessary (always on), analytics, and marketing. Preferences are persisted in localStorage with a timestamp. The banner re-appears if localStorage is cleared or on first visit.

No Secrets in Client Code

All Supabase credentials used in client components are the NEXT_PUBLIC_anon key, which is designed to be publicly visible. The service_role key does not appear anywhere in the codebase.

11. Analytics & Cookie Consent

Vercel Analytics

The `@vercel/analytics` package (v1.6.1) is integrated via the Analytics component from `@vercel/analytics/next`. It is rendered conditionally in `app/layout.tsx`: `{process.env.NODE_ENV === 'production' && }`. This ensures no analytics data is sent during local development.

Cookie Consent Implementation

The `CookieConsentBanner` component implements a full GDPR consent UI with animation (opacity fade-out on dismiss). The consent state machine has four states:

State	Meaning
pending	Banner is visible; user has not yet interacted
accepted	User clicked 'Accept All'; all categories enabled
declined	User clicked 'Reject All'; only necessary cookies
customized	User saved custom preferences via the Settings panel

Storage Key: Consent data is stored in `localStorage` under the key `'psychsupport-cookie-consent'` as a JSON object containing `{ status, preferences, timestamp }`. The component handles `localStorage` unavailability gracefully (`try/catch`).

12. Dependency Reference

Production Dependencies

Package	Version	Purpose
next	16.2.0	Core framework — App Router, Server Actions, middleware
react / react-dom	^19	React 19 with concurrent features
typescript	5.7.3	Type-safe development (devDependency)
@supabase/supabase-js	^2.105.4	Supabase client SDK
@supabase/ssr	^0.10.3	SSR-compatible cookie-based auth for Next.js
tailwindcss	^4.2.0	Utility-first CSS with CSS-first config (devDependency)
@tailwindcss/postcss	^4.2.0	PostCSS integration for Tailwind v4 (devDependency)
tw-animate-css	1.3.3	Animation plugin for Tailwind
lucide-react	^0.564.0	Icon library (~500+ icons used throughout)
@radix-ui/*	Various	25+ Radix UI primitive packages for shadcn/ui
class-variance-authority	^0.7.1	Type-safe component variant definitions
clsx + tailwind-merge	^2.1 / ^3.3	Conditional class merging (via cn() utility)
react-hook-form	^7.54.1	Performant form state management
@hookform/resolvers	^3.9.1	Zod integration for react-hook-form
zod	^3.24.1	Schema validation
date-fns	4.1.0	Date utility functions
recharts	2.15.0	Charting library (Chart component in UI kit)
sonner	^1.7.1	Toast notification system
next-themes	^0.4.6	Theme provider for dark/light mode
@vercel/analytics	1.6.1	Vercel web analytics integration
cmdk	1.1.1	Command palette primitive
embla-carousel-react	8.6.0	Carousel component
input-otp	1.4.2	OTP input component
react-day-picker	9.13.2	Calendar / date picker
react-resizable-panels	^2.1.7	Resizable panel layouts

vaul	^1.1.2	Drawer / bottom sheet component
autoprefixer	^10.4.20	CSS vendor prefix automation

Development Dependencies

Package	Version	Purpose
typescript	5.7.3	TypeScript compiler
@types/node	^22	Node.js type definitions
@types/react	^19	React type definitions
@types/react-dom	^19	ReactDOM type definitions
postcss	^8.5	CSS transformation pipeline

End of Document

PsychSupport Technical Documentation · Generated June 04, 2026